

Spring MVC Internals

Chapter 04 - Java Backend Deep-Dive Series

DispatcherServlet | HandlerMapping | Argument Resolvers | Message Converters

DispatcherServlet	Front Controller - every request enters here
HandlerMapping	URL-to-handler resolution at startup
HandlerAdapter	Bridges dispatcher to any handler type
@RequestMapping internals	How annotations register at startup
Argument Resolvers	@PathVariable, @RequestParam, @RequestBody
Message Converters	@ResponseBody -> JSON serialisation
ResponseEntity	Fine-grained HTTP response control
@ControllerAdvice	Global exception handling internals
Validation	@Valid, BindingResult, exception flow

A DispatcherServlet

The Front Controller Pattern

1. Plain English Explanation

DispatcherServlet is the single entry point for every HTTP request in a Spring MVC application. It sits in front of all controllers and orchestrates the full request processing pipeline: find the right handler, invoke it, convert the return value, and write the HTTP response. Without it, every controller would have to re-implement content-negotiation, exception handling, and view resolution.

2. Real-World Analogy

Think of a busy airport check-in hall. Every passenger (HTTP request) enters through the same front door regardless of destination. A chief dispatcher (DispatcherServlet) looks at your boarding pass (URL + HTTP method), consults the routing board (HandlerMapping), sends you to the correct desk (controller method), collects your baggage claim receipt (return value), and prints your boarding pass (HTTP response). Individual desks do not handle routing - the dispatcher does.

3. Minimal Working Example

```
// Spring Boot auto-registers DispatcherServlet via
// DispatcherServletAutoConfiguration - no XML needed.
@SpringBootApplication
public class App {
    public static void main(String[] args) {
        SpringApplication.run(App.class, args);
    }
}
// Every request to / is handled by ONE DispatcherServlet instance.
// To inspect: set logging.level.org.springframework.web=DEBUG
// You will see: DispatcherServlet - Completed 200 OK
```

4. Architecture Diagram

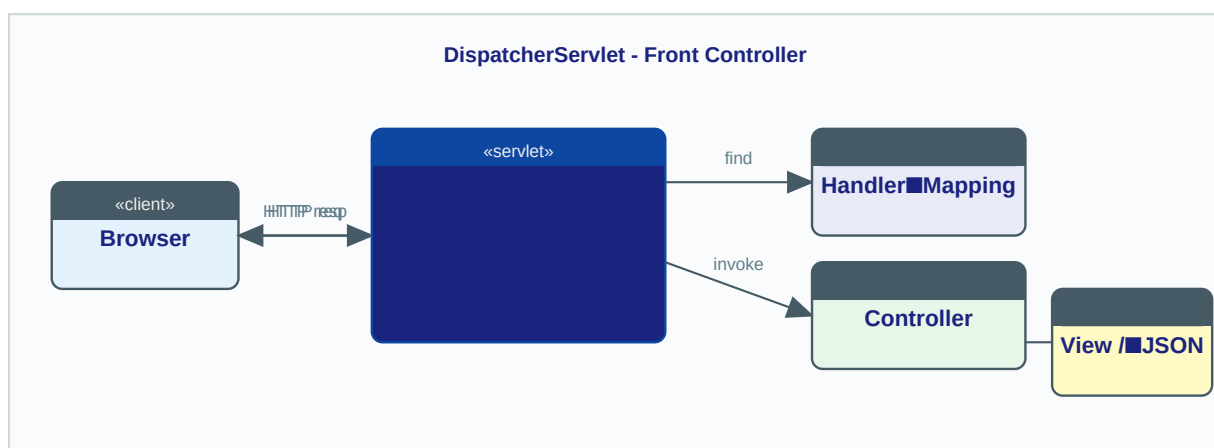


Figure A: DispatcherServlet as central front controller

5. Behind the Scenes

Key class: `org.springframework.web.servlet.DispatcherServlet`. `DispatcherServlet` extends `FrameworkServlet` which extends `HttpServlet`. The critical method is `doDispatch(HttpServletRequest, HttpServletResponse)`. Every request travels through this method.

```
// DispatcherServlet.doDispatch() - simplified pseudocode
protected void doDispatch(HttpServletRequest req,
                        HttpServletResponse resp) throws Exception {
    // 1. Find HandlerExecutionChain (handler + interceptors)
    HandlerExecutionChain chain = getHandler(req);
    // 2. Find matching HandlerAdapter
    HandlerAdapter ha = getHandlerAdapter(chain.getHandler());
    // 3. Run pre-handle interceptors
    chain.applyPreHandle(req, resp);
    // 4. Delegate actual invocation to HandlerAdapter
    ModelAndView mv = ha.handle(req, resp, chain.getHandler());
    // 5. Run post-handle interceptors
    chain.applyPostHandle(req, resp, mv);
    // 6. Render view or write response body
    processDispatchResult(req, resp, chain, mv, null);
}
```

Design pattern: Front Controller (GoF-adjacent) + Chain of Responsibility for interceptors. `DispatcherServlet` is the hub; all specialised work is delegated to strategy objects (`HandlerMapping`, `HandlerAdapter`, `ViewResolver`).

6. Realistic Practical Example

```
// Registering a second DispatcherServlet for API vs UI routing
@Bean
public ServletRegistrationBean<DispatcherServlet> apiServlet(
    WebApplicationContext ctx) {
    DispatcherServlet ds = new DispatcherServlet(ctx);
    ds.setDetectAllHandlerMappings(true);
    ServletRegistrationBean<DispatcherServlet> bean =
        new ServletRegistrationBean<>(ds, "/api/*");
    bean.setName("apiDispatcher");
    bean.setLoadOnStartup(2);
    return bean;
}
// UI requests go to default DispatcherServlet on /
// API requests go to apiDispatcher on /api/*
```

7. Common Mistakes and Misconceptions

- Thinking there is one `DispatcherServlet` per request - there is one per mapping (singleton, thread-safe).
- Forgetting that Spring Boot auto-registers it; manually registering a duplicate causes double-processing.
- Assuming `DispatcherServlet` handles `WebSocket` or `SSE` directly - it does not; use separate endpoints.
- Calling `ctx.refresh()` manually after boot - the servlet already called it during init.

B Full Request Lifecycle

HTTP Request to Response - Step by Step

1. Plain English Explanation

When a browser or REST client sends a request, Spring MVC processes it through a well-defined pipeline of 10 distinct steps. Understanding each step lets you know exactly where to hook in interceptors, add custom argument resolvers, or diagnose unexpected behaviour.

2. Real-World Analogy

Imagine a parcel moving through a courier depot. It is scanned at intake (Servlet Filter), sorted by destination (HandlerMapping), loaded onto the correct van (HandlerAdapter), opened and repackaged by the depot worker (controller method), sealed and labelled (message converter), and dispatched to the recipient (HTTP response). Each station has one job and passes the parcel on.

3. Minimal Working Example - Trace a GET /users/42

```
@RestController
@RequestMapping("/users")
public class UserController {

    @GetMapping("/{id}")
    public User getUser(@PathVariable Long id) {
        // Step 7 in lifecycle: controller method executes here.
        // Before this: filter -> DS.doDispatch -> HandlerMapping
        //                  -> interceptors -> HandlerAdapter -> ArgResolvers
        // After this: ReturnValueHandler -> MessageConverter -> response
        return userService.findById(id);
    }
}
```

4. Request Lifecycle Flow Diagram

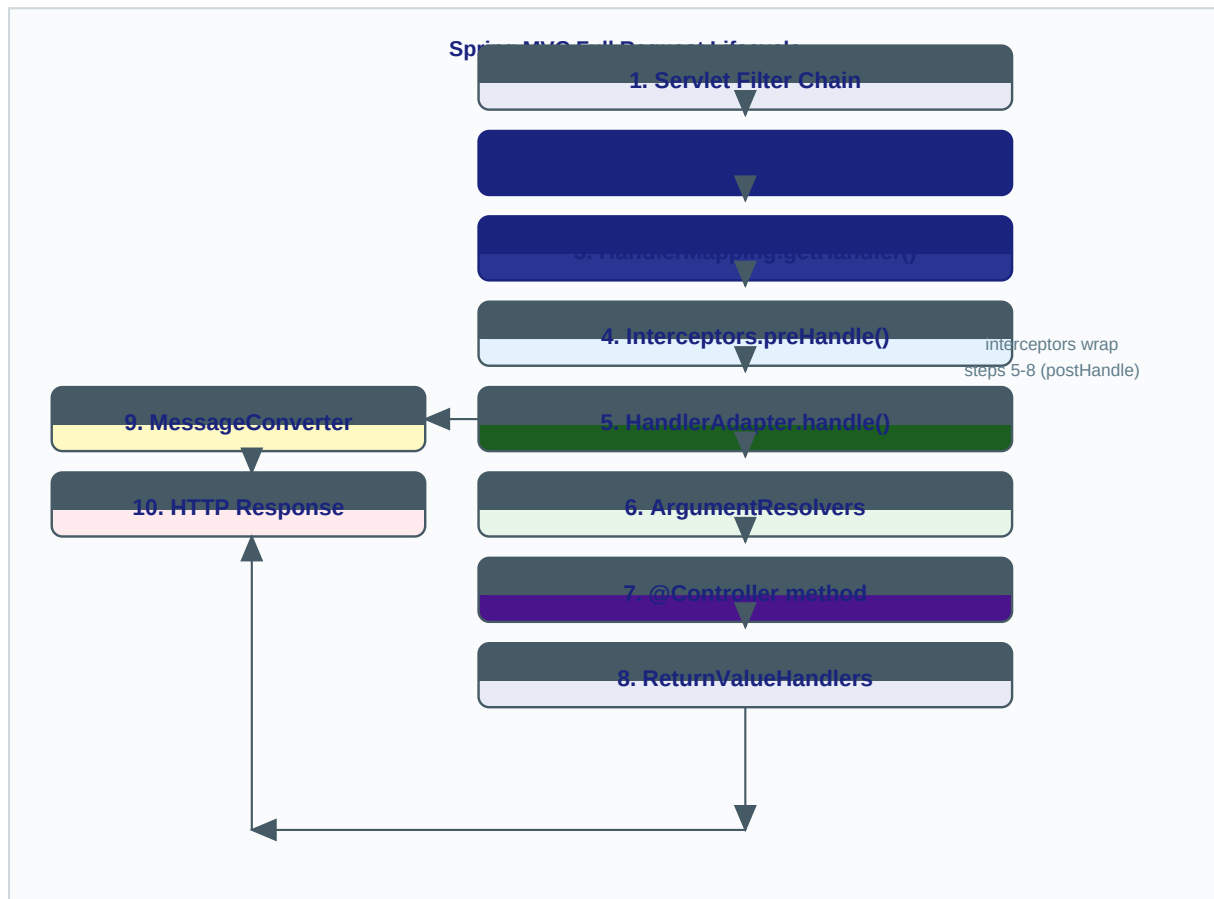


Figure B: Complete HTTP request-to-response lifecycle in Spring MVC

5. Behind the Scenes

The pipeline is driven entirely from `DispatcherServlet.doDispatch()`. Key internal calls:

```
// Step 3 - HandlerMapping lookup inside getHandler(req):
//   iterates this.handlerMappings list (ordered by @Order)
//   first mapping whose getHandler() returns non-null wins

// Step 5 - adapter selection inside getHandlerAdapter(handler):
//   for each HandlerAdapter: if (ha.supports(handler)) return ha;

// Step 8 - return value handling (inside invokeHandlerMethod):
//   HandlerMethodReturnValueHandlerComposite.handleReturnValue()
//   picks the right handler:
//   @ResponseBody -> RequestResponseBodyMethodProcessor
//   ResponseEntity -> HttpEntityMethodProcessor
//   String -> ViewNameMethodReturnValueHandler
//   Then writes to HttpServletResponse via message converter
```

Design pattern: Strategy. `DispatcherServlet` holds lists of strategies (`HandlerMapping`, `HandlerAdapter`, `ViewResolver`) and selects the right one at runtime.

6. Realistic Practical Example - Custom Interceptor

```
@Component
public class TimingInterceptor implements HandlerInterceptor {

    @Override
    public boolean preHandle(HttpServletRequest req,
```

```

        HttpServletResponse resp, Object handler) {
    req.setAttribute("startMs", System.currentTimeMillis());
    return true; // returning false halts the chain
}

@Override
public void afterCompletion(HttpServletRequest req,
    HttpServletResponse resp, Object handler, Exception ex) {
    long ms = System.currentTimeMillis()
        - (Long) req.getAttribute("startMs");
    System.out.println("Request took " + ms + " ms");
}
}

```

7. Common Mistakes and Misconceptions

- Filters (javax.servlet.Filter) run BEFORE DispatcherServlet; HandlerInterceptors run INSIDE it.
- Returning false from preHandle() stops the chain but you must write the response yourself or the client hangs.
- postHandle() is NOT called if an exception is thrown from the controller method.
- afterCompletion() IS called even after an exception - it is the correct place for cleanup and timing.

1. Plain English Explanation

HandlerMapping answers: given this URL and HTTP method, which Java method should handle it? At application startup, RequestMappingHandlerMapping scans every @Controller and @RestController bean, reads their @RequestMapping annotations, and builds an internal lookup table. At request time, the dispatcher queries this table to get back a HandlerExecutionChain containing the handler method and interceptors.

2. Real-World Analogy

HandlerMapping is a telephone directory. During application startup it is compiled once - every entry is indexed. At runtime, when a call arrives, the dispatcher looks up the number (URL pattern) in the directory to find the correct subscriber (handler method) instantly, without scanning every page.

3. Minimal Working Example

```
@RestController
public class ProductController {

    // RequestMappingHandlerMapping registers at startup:
    // GET /products -> listProducts()
    // GET /products/{id} -> getProduct(Long)
    // POST /products -> createProduct(Product)

    @GetMapping("/products")
    public List<Product> listProducts() { return List.of(); }

    @GetMapping("/products/{id}")
    public Product getProduct(@PathVariable Long id) {
        return new Product(id, "Widget");
    }

    @PostMapping("/products")
    public Product create(@RequestBody Product p) { return p; }
}
```

4. HandlerMapping Startup Diagram

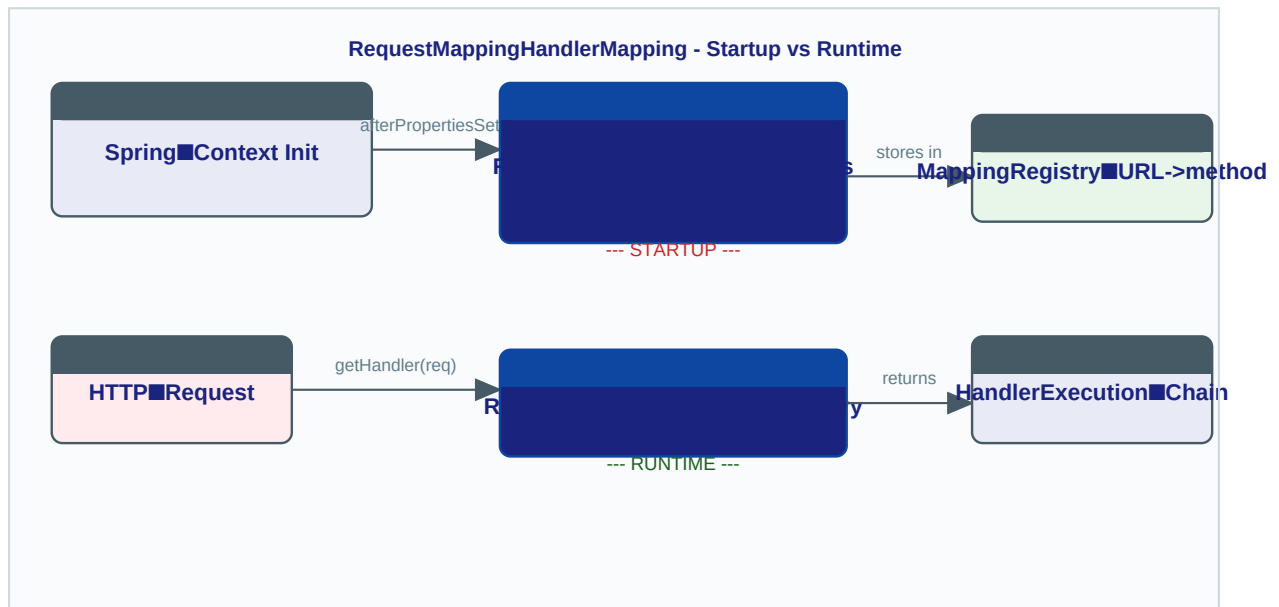


Figure C: HandlerMapping - build-once-lookup-many pattern

5. Behind the Scenes

Class: `org.springframework.web.servlet.mvc.method.annotation.RequestMappingHandlerMapping`. It implements `InitializingBean` so Spring calls `afterPropertiesSet()` during context refresh.

```
// RequestMappingHandlerMapping startup call chain:
// afterPropertiesSet()
//   -> super.afterPropertiesSet()
//   -> initHandlerMethods()
//       -> detectHandlerMethods(beanName) per @Controller
//           -> getMappingForMethod(method, handlerType)
//               builds RequestMappingInfo from @RequestMapping
//           -> registry.register(mappingInfo, handler, method)
//               stored in MappingRegistry (ConcurrentHashMap)

// At request time: getHandler(req)
//   -> lookupHandlerMethod(lookupPath, req)
//   returns HandlerExecutionChain (handler + interceptors)
```

Pattern: Registry / Lookup Table. The registry is built once and queried at $O(1)$ per request. `AntPathMatcher` or `PathPatternParser` (Spring 5.3+) handles wildcards.

6. Realistic Practical Example

```
// Switch to PathPatternParser for better performance (Spring 5.3+)
@Configuration
public class MvcConfig implements WebMvcConfigurer {

    @Override
    public void configurePathMatch(PathMatchConfigurer cfg) {
        cfg.setPatternParser(new PathPatternParser());
        // Enables {*remainder} capture groups
        // Faster than AntPathMatcher for complex patterns
    }
}

// Spring Boot 2.6+ uses PathPatternParser by default.
```

7. Common Mistakes and Misconceptions

- Duplicate URL mappings cause `IllegalStateException` at STARTUP, not at request time.
- Without `@EnableWebMvc` (or Spring Boot auto-config), RMHM is NOT registered automatically.
- `SimpleUrlHandlerMapping` and `BeanNameUrlHandlerMapping` are older alternatives; RMHM is the default.
- Order matters: multiple `HandlerMappings` are tried in `@Order` sequence; the first non-null result wins.

D HandlerAdapter

Why It Exists - RequestMappingHandlerAdapter

1. Plain English Explanation

DispatcherServlet does not invoke controllers directly. It asks: which HandlerAdapter can invoke THIS particular handler? HandlerAdapter is an abstraction that allows different types of controllers (annotated methods, HttpRequestHandler, legacy Controller interface) to coexist. RequestMappingHandlerAdapter handles the @RequestMapping style used in almost every modern application.

2. Real-World Analogy

A HandlerAdapter is a universal power adapter. Your laptop (DispatcherServlet) outputs a standard USB-C interface. The adapter (HandlerAdapter) converts that to whichever local socket standard the target device (controller) uses. Swap the adapter and you can power completely different devices without changing the laptop.

3. Minimal Working Example

```
// Spring internally does this check in DispatcherServlet:
for (HandlerAdapter adapter : this.handlerAdapters) {
    if (adapter.supports(handler)) {
        // RequestMappingHandlerAdapter returns true for
        // HandlerMethod instances (annotated controller methods)
        return adapter;
    }
}
// Once found:
ModelAndView mv = ha.handle(request, response, handler);
// For @RestController methods mv is null (response already written)
```

4. HandlerAdapter Adapter Pattern Diagram

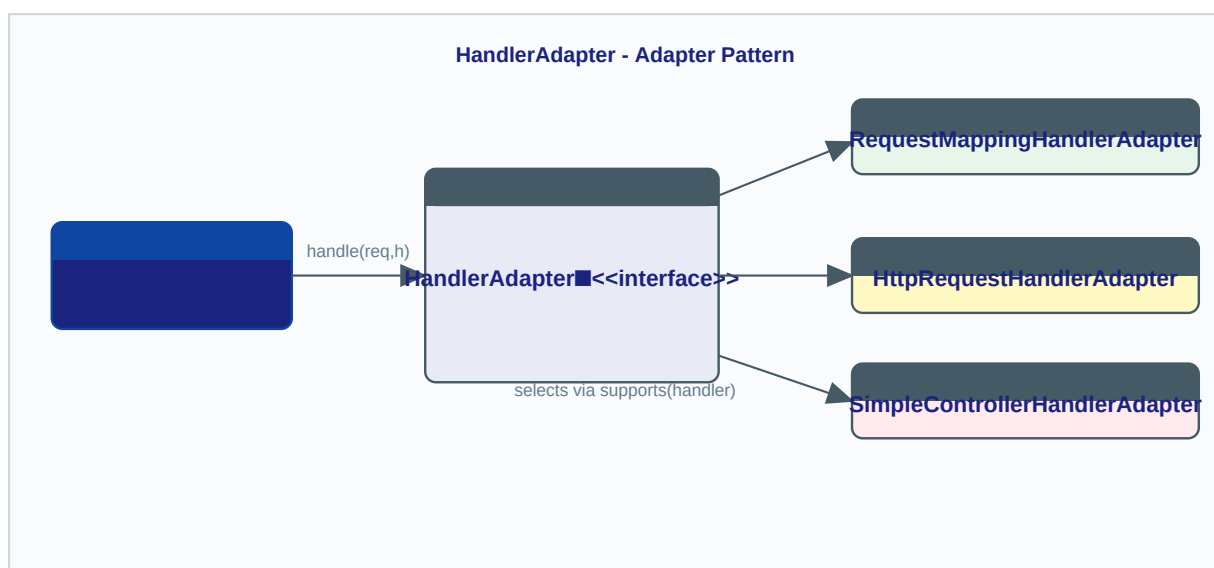


Figure D: HandlerAdapter - Adapter pattern isolates DispatcherServlet from handler types

5. Behind the Scenes

Class: `org.springframework.web.servlet.mvc.method.annotation.RequestMappingHandlerAdapter`. The real work happens in `invokeHandlerMethod()`:

```
// RequestMappingHandlerAdapter.invokeHandlerMethod() key steps:
ServletInvocableHandlerMethod invocable =
    createInvocableHandlerMethod(handlerMethod);

// Attach all ArgumentResolvers
invocable.setHandlerMethodArgumentResolvers(
    this.argumentResolvers); // HandlerMethodArgumentResolverComposite

// Attach all ReturnValueHandlers
invocable.setHandlerMethodReturnValueHandlers(
    this.returnValueHandlers); // HandlerMethodReturnValueHandlerComposite

// Actually invoke the @Controller method
invocable.invokeAndHandle(webRequest, mavContainer);
// Result written to ModelAndViewContainer or directly to response
```

Pattern: Adapter (GoF). `RequestMappingHandlerAdapter` adapts the generic `handle(request, response, handler)` contract to annotated method invocation.

6. Realistic Practical Example

```
// Register a custom HandlerAdapter for a legacy handler type
@Configuration
public class CustomAdapterConfig {

    @Bean
    public HandlerAdapter legacyAdapter() {
        return new HttpRequestHandlerAdapter();
        // Handles beans implementing HttpRequestHandler directly
    }

    @Bean("/legacy-handler")
    public HttpRequestHandler legacyHandler() {
        return (req, resp) ->
            resp.getWriter().write("legacy handler ok");
    }
}
// Navigating to /legacy-handler invokes legacyHandler.handleRequest()
```

7. Common Mistakes and Misconceptions

- `RequestMappingHandlerAdapter` must be registered as a bean - `@EnableWebMvc` or Spring Boot does this.
- The adapter does not directly invoke the method - it delegates to `ServletInvocableHandlerMethod`.
- `HandlerAdapter` INVOKES the handler; `HandlerMapping` only FINDS it - they are completely separate concerns.
- Registering a `@Bean` of type `HandlerAdapter` replaces the default list; use `WebMvcConfigurer` to customise instead.

1. Plain English Explanation

@RequestMapping (and its shortcuts @GetMapping, @PostMapping, etc.) are pure metadata annotations. They do nothing at compile time. During context startup, RequestMappingHandlerMapping reads them via reflection and builds an in-memory registry mapping each (URL pattern + HTTP method + headers + consumes + produces) tuple to a specific Java Method object.

2. Real-World Analogy

Think of annotations as sticky notes on hotel room doors reading 'Room 101 - check-in desk, English speakers only'. The hotel manager (Spring startup) reads all sticky notes once, writes them in the master booking system (MappingRegistry), and tears the notes down. Guests (requests) are routed via the booking system - they never see the original sticky notes.

3. Minimal Working Example

```
@RestController
@RequestMapping("/api/v1") // class-level prefix
public class OrderController {

    // Registered: GET /api/v1/orders
    @GetMapping("/orders")
    public List<Order> list() { return List.of(); }

    // Registered: POST /api/v1/orders (consumes JSON)
    @PostMapping(value = "/orders",
        consumes = "application/json")
    public Order create(@RequestBody Order o) { return o; }

    // Registered: DELETE /api/v1/orders/{id}
    @DeleteMapping("/orders/{id}")
    public void delete(@PathVariable Long id) {}
}
```

4. Registration Flow Diagram

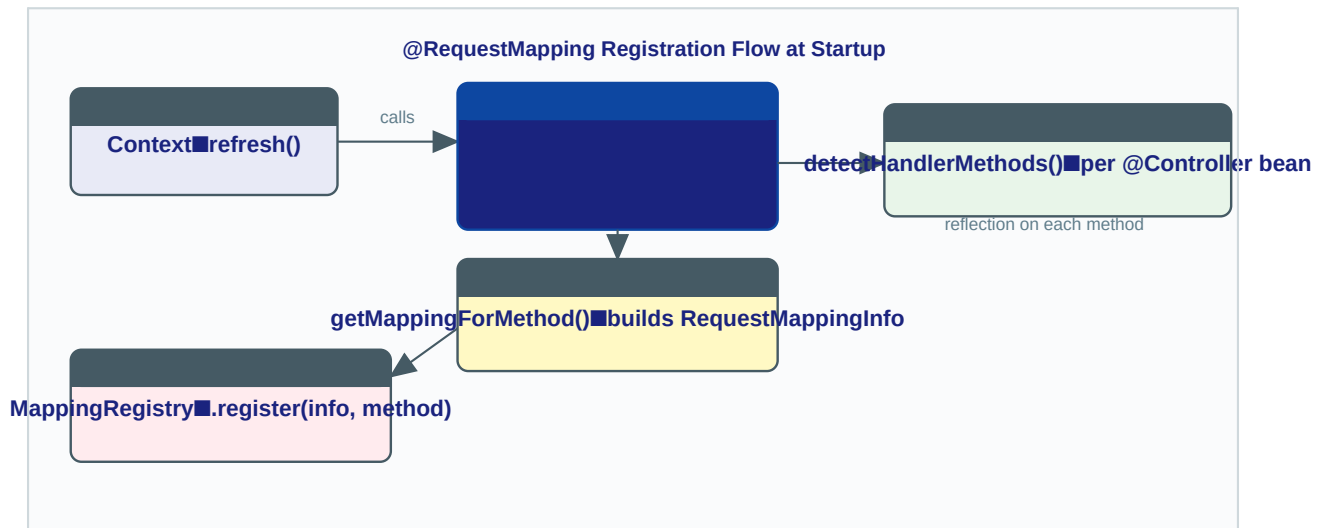


Figure E: @RequestMapping annotation processing at context startup

5. Behind the Scenes

```
// RequestMappingHandlerMapping.getMappingForMethod() internals:
RequestMappingInfo methodInfo =
    createRequestMappingInfo(
        method.getAnnotation(RequestMapping.class), null);
RequestMappingInfo typeInfo =
    createRequestMappingInfo(
        handlerType.getAnnotation(RequestMapping.class), null);
// Combine class-level prefix + method-level into one info object
return (typeInfo != null)
    ? typeInfo.combine(methodInfo) : methodInfo;

// @GetMapping is @RequestMapping(method=GET) as a meta-annotation.
// Spring reads it via AnnotatedElementUtils.findMergedAnnotation()
// which traverses the annotation hierarchy transparently.
```

Design note: @GetMapping is a composed annotation (meta-annotated with @RequestMapping). Spring's AnnotationUtils handles the meta-annotation indirection transparently.

6. Realistic Practical Example

```
// Header-conditional mapping
@GetMapping(value = "/feature",
    headers = "X-Feature-Flag=beta")
public String betaFeature() { return "beta enabled"; }

// Produces-based content negotiation
@GetMapping(value = "/data", produces = "application/json")
public Data jsonData() { return new Data(); }

@GetMapping(value = "/data", produces = "application/xml")
public Data xmlData() { return new Data(); }
// Spring picks based on the request Accept header at runtime.
// Both are registered as distinct mappings at startup.
```

7. Common Mistakes and Misconceptions

- @GetMapping is NOT processed at compile time - only at Spring startup via reflection.

- Adding `@RequestMapping` to a non-`@Controller` bean silently does nothing.
- Class-level `@RequestMapping` prefix does NOT apply to `@ExceptionHandler` methods in the same class.
- Subtle differences in consumes/headers can allow duplicate URL patterns that create ambiguous mappings.

1. Plain English Explanation

When Spring invokes your controller method it must supply values for every parameter. Argument resolvers are the components that extract those values. `HandlerMethodArgumentResolverComposite` holds a list of resolvers and picks the one that supports each parameter. `@PathVariable` is resolved from the URL path, `@RequestParam` from the query string or form data, and `@RequestBody` by deserialising the request body using an `HttpMessageConverter`.

2. Real-World Analogy

Imagine a chef (controller method) needing three ingredients: a vegetable from the garden (`@PathVariable`), a spice from the pantry shelf (`@RequestParam`), and a dish that arrived pre-cooked from a supplier (`@RequestBody`). Each ingredient comes from a different source and requires a different extraction process. The sous-chef (`ArgumentResolverComposite`) knows exactly which storage room to raid for each item.

3. Minimal Working Example

```
@RestController
@RequestMapping("/items")
public class ItemController {

    // @PathVariable - from URI template variable {id}
    // @RequestParam - from query string ?page=2&size=10
    // @RequestBody - deserialised from JSON request body
    @PutMapping("/{id}")
    public Item update(
        @PathVariable Long id,
        @RequestParam(defaultValue = "false") boolean notify,
        @RequestBody Item item) {
        item.setId(id);
        return item;
    }
}
```

4. Argument Resolution Diagram

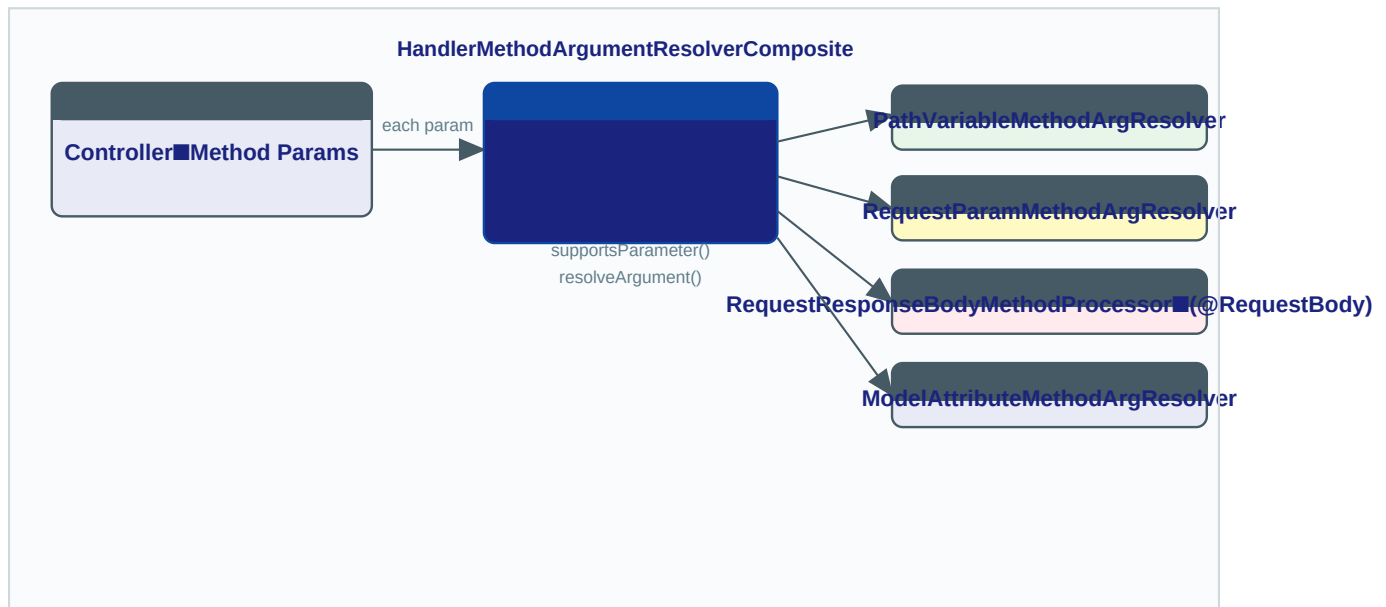


Figure F: Argument resolver chain - composite selects via supportsParameter()

5. Behind the Scenes

```

// HandlerMethodArgumentResolverComposite.resolveArgument():
public Object resolveArgument(MethodParameter param,
    ModelAndViewContainer mav, NativeWebRequest req,
    WebDataBinderFactory binderFactory) throws Exception {
    HandlerMethodArgumentResolver resolver =
        getArgumentResolver(param); // cached after first call
    return resolver.resolveArgument(param, mav, req, binderFactory);
}

// @RequestBody path - RequestResponseBodyMethodProcessor:
// -> readWithMessageConverters(req, param, targetType)
// iterates HttpMessageConverter list; first converter where
// canRead(targetType, mediaType) is true wins.
// MappingJackson2HttpMessageConverter wins for JSON.
// Then: objectMapper.readValue(inputStream, targetType)

```

Design pattern: Chain of Responsibility. Each resolver answers supportsParameter(param); the first returning true is used. The result is cached in a ConcurrentHashMap keyed by MethodParameter after the first resolution.

6. Realistic Practical Example - Custom Argument Resolver

```

@Component
public class TenantArgumentResolver
    implements HandlerMethodArgumentResolver {

    @Override
    public boolean supportsParameter(MethodParameter p) {
        return p.getParameterAnnotation(CurrentTenant.class) != null;
    }

    @Override
    public Object resolveArgument(MethodParameter p,
        ModelAndViewContainer mav, NativeWebRequest req,
        WebDataBinderFactory factory) {
        String jwt = req.getHeader("Authorization");
    }
}

```

```
        return jwtService.extractTenant(jwt);
    }
}
// Register in WebMvcConfigurer.addArgumentResolvers()
```

7. Common Mistakes and Misconceptions

- @RequestParam throws MissingServletRequestParameterException if required=true (default) and absent.
- @PathVariable name must match the URI template variable name exactly (case-sensitive).
- @RequestBody reads the InputStream once; a second read in a filter requires a wrapper that caches the body.
- Custom resolvers via addArgumentResolvers() run AFTER built-ins; to override, use RequestMappingHandlerAdapter.setArgumentResolvers().

G @ResponseBody and Message Converters

Java Objects to JSON - MappingJackson2HttpMessageConverter

1. Plain English Explanation

@ResponseBody tells Spring: do not look for a view template - write the return value directly to the HTTP response body. RequestResponseBodyMethodProcessor delegates to the registered HttpMessageConverter list. For JSON, MappingJackson2HttpMessageConverter uses Jackson's ObjectMapper to serialise the Java object. Content-Type negotiation (Accept header vs produces attribute) selects which converter is used.

2. Real-World Analogy

Imagine a translator service. Your controller speaks Java object. The client speaks JSON. @ResponseBody is the instruction 'please translate this'. MappingJackson2HttpMessageConverter is the translator who knows both languages. If the client requested XML instead, a different translator (Jaxb2RootElementHttpMessageConverter) would be chosen based on the Accept header.

3. Minimal Working Example

```
@RestController // = @Controller + @ResponseBody on every method
public class GreetController {

    // Return value serialised to JSON automatically
    @GetMapping("/greet")
    public Map<String, String> greet() {
        return Map.of("message", "Hello", "status", "ok");
    }
    // Jackson serialises -> {"message":"Hello","status":"ok"}
    // Content-Type: application/json set automatically
    // HTTP 200 OK returned (default)
}
```

4. Message Converter Selection Diagram

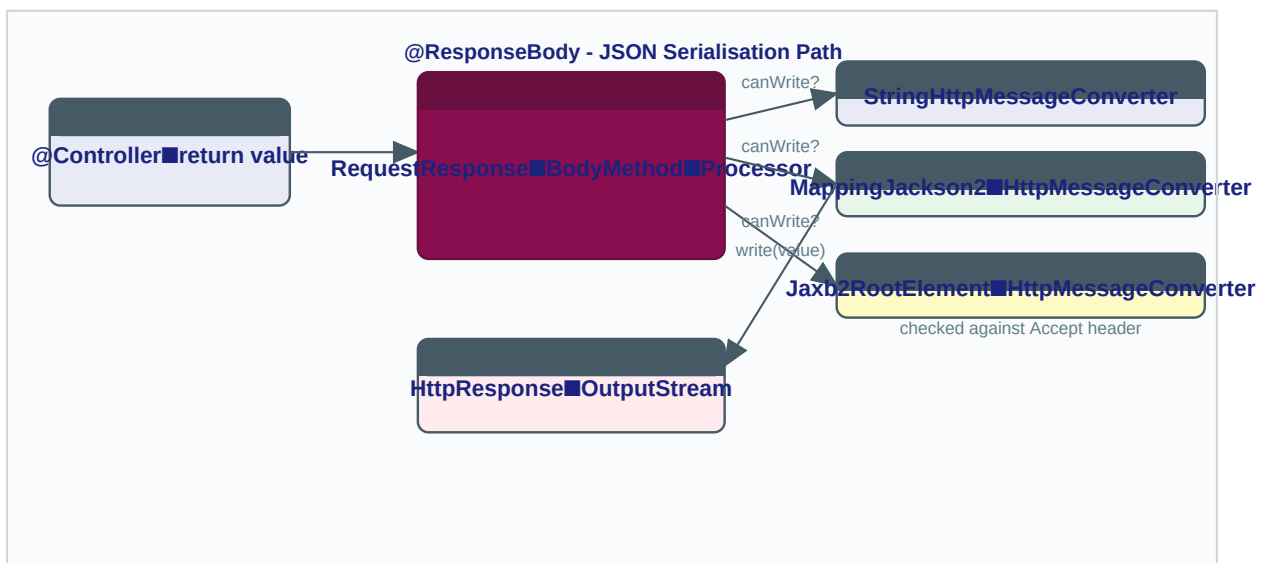


Figure G: Message converter selection - first canWrite() match wins

5. Behind the Scenes

```
// AbstractMessageConverterMethodProcessor.writeWithMessageConverters()
for (HttpMessageConverter converter : messageConverters) {
    if (converter instanceof GenericHttpMessageConverter) {
        GenericHttpMessageConverter gc =
            (GenericHttpMessageConverter) converter;
        if (gc.canWrite(valueType, targetType, selectedMediaType)) {
            // MappingJackson2HttpMessageConverter wins for JSON
            gc.write(value, targetType,
                selectedMediaType, outputMessage);
            return;
        }
    }
}
// MappingJackson2HttpMessageConverter.writeInternal():
// objectMapper.writeValue(outputStream, value)
// Sets Content-Type: application/json on the response.
```

Pattern: Strategy. Each converter is a strategy for a particular media type. Content negotiation (Accept header vs produces attribute) determines which strategy wins. `@RestController` is a meta-annotation combining `@Controller` + `@ResponseBody`.

6. Realistic Practical Example - Custom ObjectMapper

```
@Configuration
public class JacksonConfig {

    @Bean
    public Jackson2ObjectMapperBuilderCustomizer customizer() {
        return builder -> builder
            .featuresToDisable(
                SerializationFeature.WRITE_DATES_AS_TIMESTAMPS)
            .modules(new JavaTimeModule())
            .serializationInclusion(JsonInclude.Include.NON_NULL);
        // All @RestController responses use this ObjectMapper.
        // NON_NULL means null fields are omitted from JSON output.
    }
}
```

7. Common Mistakes and Misconceptions

- `@ResponseBody` is redundant inside `@RestController` - `@RestController` already carries it as a meta-annotation.
- If no converter `canWrite()` the return type, `HttpMediaTypeNotAcceptableException` (406) is thrown.
- `configureMessageConverters()` REPLACES the default list; use `extendMessageConverters()` to ADD a converter.
- Jackson serialises fields by default; null fields are included unless configured with `Include.NON_NULL`.

1. Plain English Explanation

ResponseEntity is a wrapper that lets a controller method control the exact HTTP status code, response headers, and response body. Without it, Spring always returns 200 OK. ResponseEntity gives you full programmatic control over all three components of the HTTP response.

2. Real-World Analogy

Returning a plain object from a controller is like handing someone a book with no wrapping. ResponseEntity is a courier box where you choose the box colour (status code), stick custom labels on it (headers), and put the book inside (body). The recipient gets all three pieces of information together as a single package.

3. Minimal Working Example

```
@RestController
@RequestMapping("/docs")
public class DocController {

    @PostMapping
    public ResponseEntity<Doc> create(@RequestBody Doc doc) {
        Doc saved = service.save(doc);
        URI location = URI.create("/docs/" + saved.getId());
        return ResponseEntity
            .created(location)          // 201 Created + Location header
            .header("X-Doc-Id", saved.getId().toString())
            .body(saved);              // response body = serialised Doc
    }

    @DeleteMapping("/{id}")
    public ResponseEntity<Void> delete(@PathVariable Long id) {
        service.delete(id);
        return ResponseEntity.noContent().build(); // 204 No Content
    }
}
```

4. ResponseEntity Internals Diagram

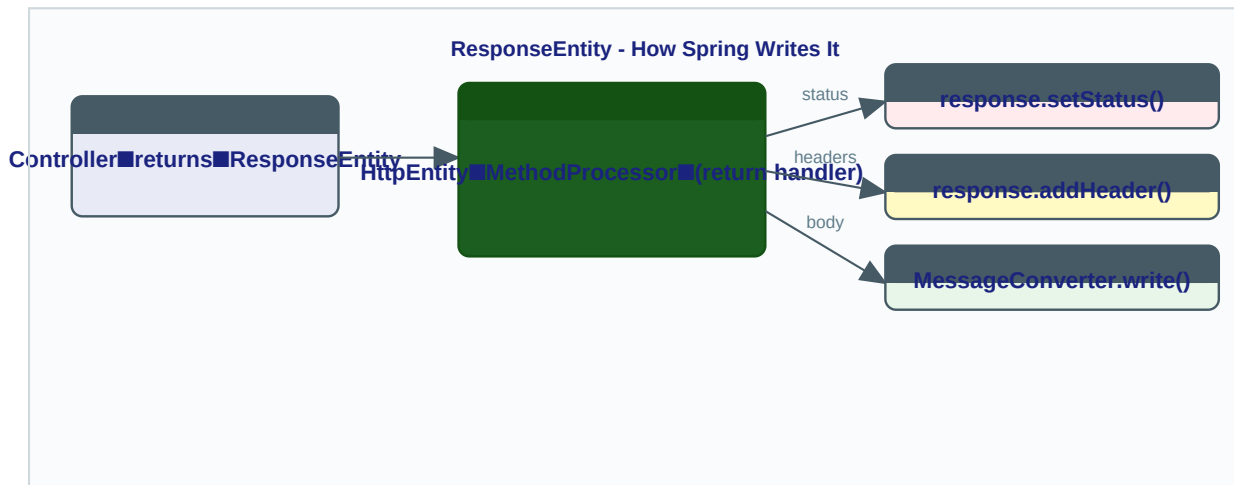


Figure H: `HttpEntityMethodProcessor` extracts status, headers, and body from `ResponseEntity`

5. Behind the Scenes

```
// HttpEntityMethodProcessor.handleReturnValue() - simplified
HttpEntity<?> httpEntity = (HttpEntity<?>) returnValue;
if (httpEntity instanceof ResponseEntity) {
    ResponseEntity<?> re = (ResponseEntity<?>) httpEntity;
    // Apply HTTP status code
    outputMessage.setStatusCode(re.getStatusCode());
}
// Copy all response headers
HttpHeaders headers = httpEntity.getHeaders();
outputMessage.getHeaders().putAll(headers);
// Write the body using message converters (same path as @ResponseBody)
Object body = httpEntity.getBody();
if (body != null) {
    writeWithMessageConverters(body, returnType,
                               inputMessage, outputMessage);
}
```

6. Realistic Practical Example - Conditional GET with ETag

```
@GetMapping("/{id}")
public ResponseEntity<Product> get(
    @PathVariable Long id,
    @RequestHeader(value="If-None-Match", required=false)
    String etag) {
    Product p = service.find(id);
    String currentEtag = "'" + p.getVersion() + "'";
    if (currentEtag.equals(etag)) {
        return ResponseEntity.status(HttpStatus.NOT_MODIFIED)
            .eTag(currentEtag).build();
    }
    return ResponseEntity.ok().eTag(currentEtag).body(p);
}
```

7. Common Mistakes and Misconceptions

- `ResponseEntity.ok(body)` and `ResponseEntity.ok().body(body)` are equivalent - both return 200.
- Setting Content-Type in `ResponseEntity` headers is overridden by the message converter - do not set it manually.

- ResponseEntity with .build() produces no body and is correct for 204; returning null gives 200 with empty body.
- Using ResponseEntity defeats Jackson type inference - prefer strongly typed generics like ResponseEntity.

1. Plain English Explanation

@ExceptionHandler marks a method that handles specific exception types thrown by controller methods. @ControllerAdvice applies those handlers globally across all controllers. Internally,ExceptionHandlerExceptionResolver discovers all @ControllerAdvice beans at startup and builds a lookup table. When an exception escapes a controller, DispatcherServlet catches it and delegates to this resolver.

2. Real-World Analogy

An airline has a customer service team (ExceptionHandlerExceptionResolver). @ControllerAdvice is the call centre handling complaints from ANY flight (controller). @ExceptionHandler methods are specialists: one team handles lost luggage (EntityNotFoundException), another handles overbooking (ConflictException). DispatcherServlet is the gate agent who transfers angry passengers to the right team.

3. Minimal Working Example

```
@RestControllerAdvice // = @ControllerAdvice + @ResponseBody
public class GlobalExceptionHandler {

    @ExceptionHandler(EntityNotFoundException.class)
    @ResponseStatus(HttpStatus.NOT_FOUND)
    public ErrorResponse handleNotFound(EntityNotFoundException ex) {
        return new ErrorResponse("NOT_FOUND", ex.getMessage());
    }

    @ExceptionHandler(MethodArgumentNotValidException.class)
    @ResponseStatus(HttpStatus.BAD_REQUEST)
    public ErrorResponse handleValidation(
        MethodArgumentNotValidException ex) {
        String msg = ex.getBindingResult().getAllErrors().stream()
            .map(DefaultMessageSourceResolvable::getDefaultMessage)
            .collect(Collectors.joining(", "));
        return new ErrorResponse("VALIDATION_FAILED", msg);
    }
}
```

4. Exception Handling Flow Diagram

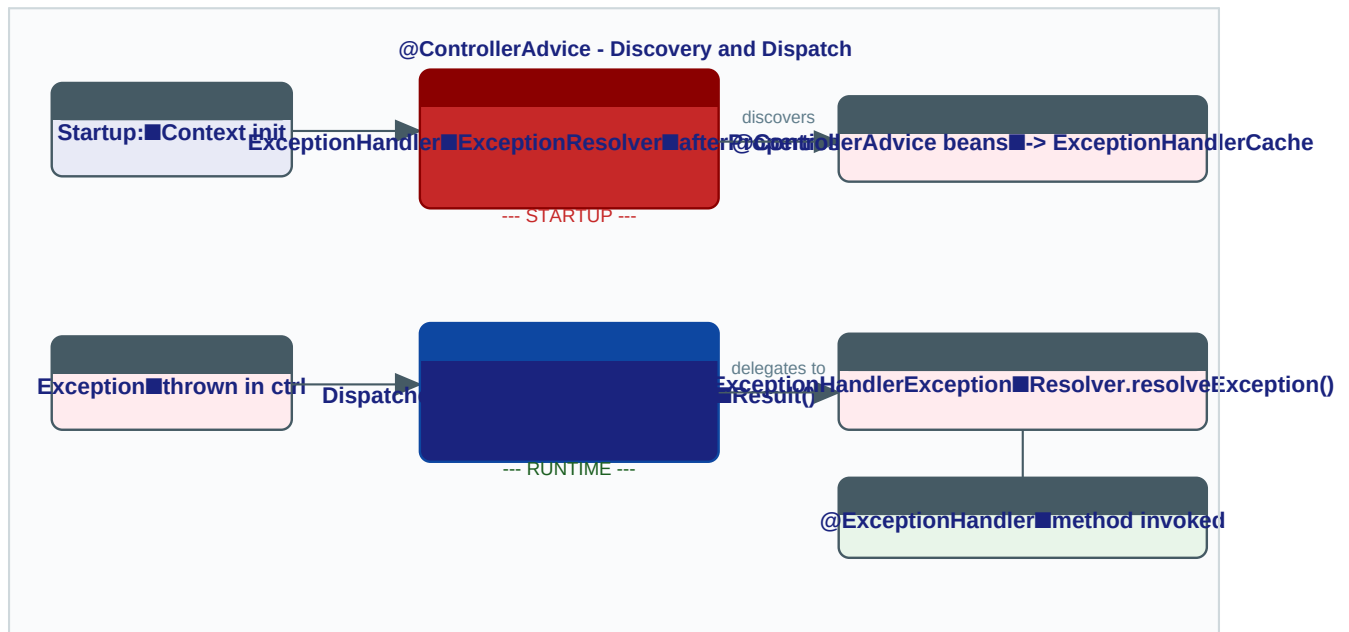


Figure 1: @ControllerAdvice discovery at startup; exception routing at runtime

5. Behind the Scenes

```
// ExceptionHandlerExceptionHandlerResolver.afterPropertiesSet():
// 1. Finds all beans annotated with @ControllerAdvice
// 2. For each, creates ExceptionHandlerMethodResolver
//    which maps ExceptionType -> @ExceptionHandler Method
// 3. Stores in LinkedHashMap (ordered, global advice evaluated first)

// At runtime: resolveException(req, resp, handler, ex)
// 1. Try handler-local @ExceptionHandler first (same controller)
// 2. Then try each @ControllerAdvice bean in @Order order
// 3. Match by: exact type, then most-specific supertype
// 4. Invoke via ServletInvocableHandlerMethod - same pipeline
//    as controller method: argument resolvers + return value handlers
```

Pattern: Chain of Responsibility. Exception resolvers are tried in precedence order. `ExceptionHandlerExceptionHandlerResolver` has the highest precedence among built-in resolvers. `@ControllerAdvice` beans are ordered by `@Order / Ordered`.

6. Realistic Practical Example

```
@RestControllerAdvice
@Order(Ordered.HIGHEST_PRECEDENCE)
public class SecurityExceptionHandler {

    @ExceptionHandler(AccessDeniedException.class)
    public ResponseEntity<ErrorResponse> handleAccessDenied(
        AccessDeniedException ex, HttpServletRequest req) {
        log.warn("Access denied: {} {}",
            req.getMethod(), req.getRequestURI());
        return ResponseEntity
            .status(HttpStatus.FORBIDDEN)
            .body(new ErrorResponse("FORBIDDEN", ex.getMessage()));
    }
}

// @Order(HIGHEST_PRECEDENCE) ensures this runs before a generic handler
```

7. Common Mistakes and Misconceptions

- `@ExceptionHandler` does NOT catch exceptions thrown by Servlet Filters - filters run before `DispatcherServlet`.
- A local `@ExceptionHandler` (same `@Controller`) takes priority over `@ControllerAdvice` for that controller.
- `@ControllerAdvice` without specifiers applies to ALL controllers; use `basePackages` to narrow scope.
- Returning `ResponseEntity` from `@ExceptionHandler` while also using `@ResponseStatus`: `ResponseEntity` wins.

1. Plain English Explanation

Adding `@Valid` (or `@Validated`) before a `@RequestBody` or `@ModelAttribute` parameter tells Spring to run Bean Validation (JSR-380) on the incoming object. If the parameter is immediately followed by a `BindingResult`, validation errors are captured there and the method is still invoked. If no `BindingResult` is present, Spring throws `MethodArgumentNotValidException` on any constraint violation.

2. Real-World Analogy

Validation is like a hotel concierge who checks your reservation details before escorting you to your room. If your name is missing (`@NotBlank`), he can either hand you a clipboard to note the issue and still let you in (`BindingResult`), or refuse entry entirely and send you back to the front desk (`MethodArgumentNotValidException`). You choose the policy in how you write the method.

3. Minimal Working Example

```
public class RegisterRequest {
    @NotBlank(message = "Username is required")
    private String username;

    @Email(message = "Valid email required")
    private String email;

    @Size(min = 8, message = "Password min 8 chars")
    private String password;
    // getters/setters omitted for brevity
}

@PostMapping("/register")
public ResponseEntity<String> register(
    @Valid @RequestBody RegisterRequest req,
    BindingResult result) {    // MUST immediately follow @Valid
    if (result.hasErrors()) {
        return ResponseEntity.badRequest()
            .body(result.getAllErrors().toString());
    }
    return ResponseEntity.ok("Registered: " + req.getUsername());
}
```

4. Validation Flow Diagram

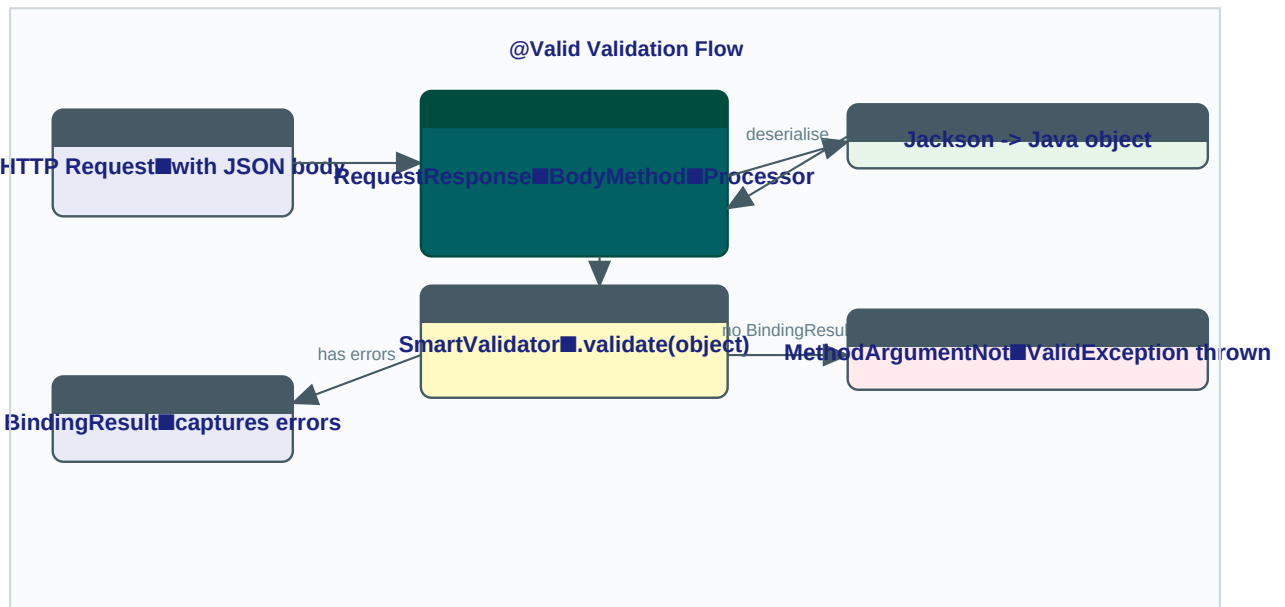


Figure J: @Valid processing - BindingResult path vs exception path

5. Behind the Scenes

```

// RequestResponseBodyMethodProcessor.resolveArgument() - after
// reading body with message converter, validation is triggered:
if (binder.getTarget() != null) {
    validateIfApplicable(binder, parameter);
    // validateIfApplicable checks for @Valid / @Validated
    // calls LocalValidatorFactoryBean -> Hibernate Validator
    // which traverses constraints on the object graph
}
if (binder.getBindingResult().hasErrors()
    && isBindExceptionRequired(binder, parameter)) {
    // isBindExceptionRequired returns TRUE if next param
    // is NOT a BindingResult or Errors type
    throw new MethodArgumentNotValidException(
        parameter, binder.getBindingResult());
}
// MethodValidationInterceptor handles @Validated on class/method
// level via Spring AOP proxy (requires @EnableMethodValidation).
  
```

Pattern: Template Method. `validateIfApplicable()` is a hook in the argument resolver base class. Hibernate Validator (reference implementation) traverses the object graph following `@Valid` on nested fields for cascaded validation.

6. Realistic Practical Example - Global Validation Handler

```

@RestControllerAdvice
public class ValidationAdvice {

    @ExceptionHandler(MethodArgumentNotValidException.class)
    @ResponseStatus(HttpStatus.UNPROCESSABLE_ENTITY)
    public Map<String, String> handleValidation(
        MethodArgumentNotValidException ex) {
        Map<String, String> errors = new LinkedHashMap<>();
        ex.getBindingResult().getFieldErrors().forEach(e ->
            errors.put(e.getField(), e.getDefaultMessage()));
        return errors;
        // {"username":"Username is required","email":"Valid email required"}
    }
}
  
```

```
    }  
}  
// No BindingResult in controllers needed - exceptions handled globally
```

7. Common Mistakes and Misconceptions

- @Valid is JSR-380 (standard); @Validated is Spring-specific and enables group validation.
- BindingResult MUST immediately follow the validated parameter - any gap causes Spring to throw instead of capturing.
- @Validated on @Service requires Spring AOP proxy via @EnableMethodValidation - not automatic.
- Nested object validation requires @Valid on the field inside the parent class, not just on the controller parameter.

Spring MVC Internals - Quick Reference

Component / Method	Role in Spring MVC
DispatcherServlet.doDispatch()	Entry point for every request; orchestrates the full pipeline
HandlerMapping.getHandler(req)	Returns HandlerExecutionChain (handler + interceptors)
RequestMappingHandlerMapping	Builds URL-to-HandlerMethod map via afterPropertiesSet()
HandlerAdapter.handle()	Invokes the handler; RMHA delegates to ServletInvocableHandlerMethod
HandlerMethodArgumentResolver	Extracts one method parameter; composite selects by supportsParameter()
RequestResponseBodyMethodProcessor	Handles @RequestBody (read) and @ResponseBody (write)
HttpMessageConverter.write()	Serialises Java object to response body (Jackson for JSON)
HttpEntityMethodProcessor	Handles ResponseEntity return values; sets status and headers
ExceptionHandlerExceptionResolver	Discovers @ControllerAdvice at startup; dispatches on exception
MethodArgumentNotValidException	Thrown by @Valid when no BindingResult parameter follows
HandlerMethodReturnValueHandlerComposite	Picks correct ReturnValueHandler for method return type
MappingRegistry (ConcurrentHashMap)	Stores URL-pattern -> HandlerMethod after startup annotation scan

Design Patterns in Spring MVC

Pattern	Where Used	Purpose
Front Controller	DispatcherServlet	Single entry point routes all requests
Strategy	HandlerMapping, HandlerAdapter, MessageConverter	Interchangeable algorithms selected at runtime
Chain of Responsibility	HandlerInterceptor, ExceptionResolver	Each element handles or passes on the call
Adapter	HandlerAdapter	Adapts diverse handler types to uniform handle() contract

Template Method	validateIfApplicable() in base resolver	Hook points for subclass customisation
Registry	MappingRegistry	Build-once, query-many lookup of URL to handler
Composite	ArgumentResolverComposite, ReturnValueHandlerComposite	Delegates to a list of strategies